

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

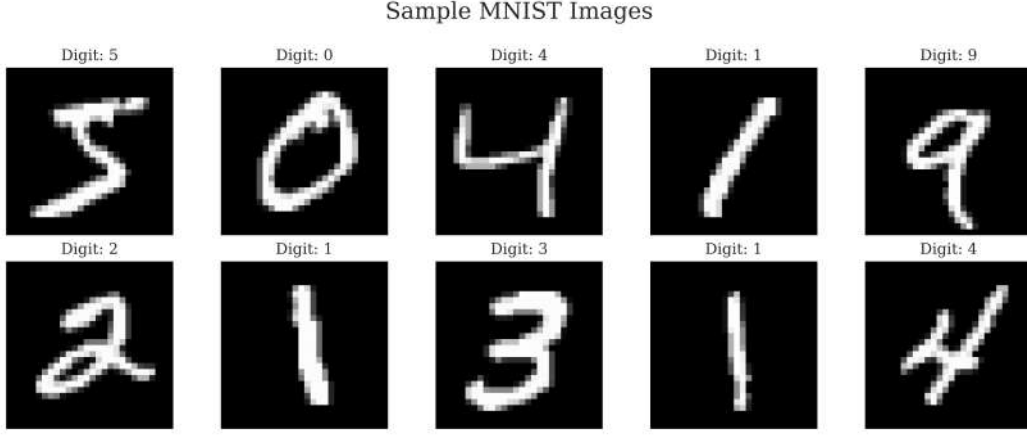
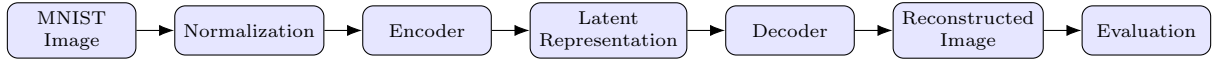


Figure 1: MNIST sample images used in the experiment.

Execution finding. MNIST images were standardized to have a size of $28 \times 28 \times 1$. The 2,000 images from the test set were saved for the final testing, while the 10,000 images from the laboratory subset of training data were further divided into 9,000 images used for training and 1,000 for validation.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

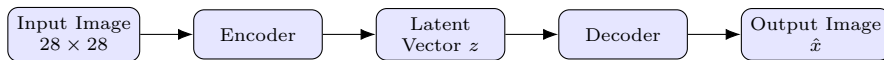
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

Required inference: Determine whether the model converges and whether there is evidence of overfitting.

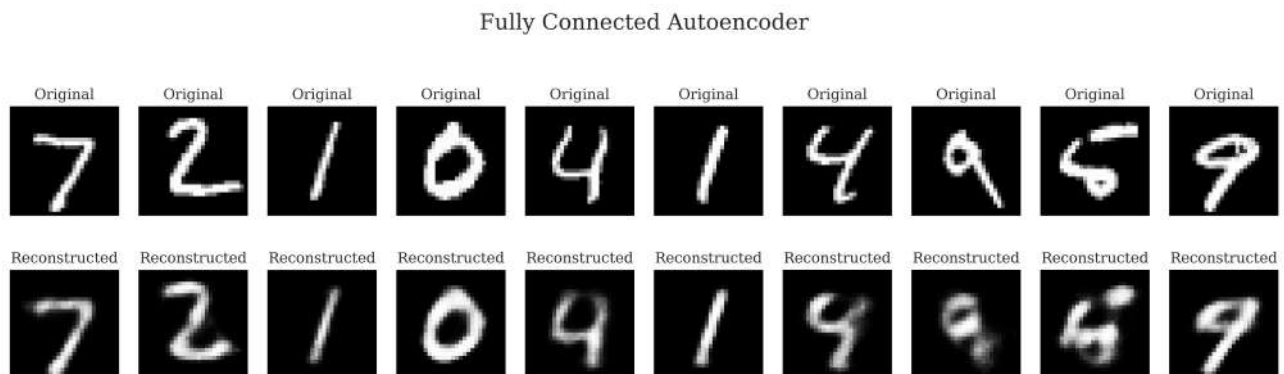


Figure 2: Original test images and their fully connected autoencoder reconstructions.

Execution finding. The digit reconstructions retain the overall structure and essence of most of the digits, although finer features do not get captured as accurately as through the convolutional model.

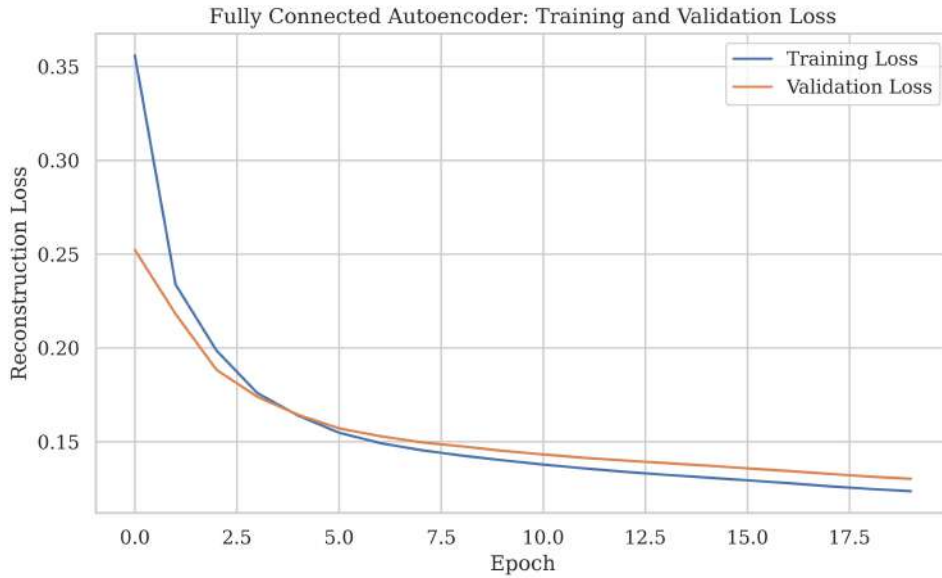


Figure 3: Training and validation loss for the fully connected autoencoder.

Inference from the execution. The loss during training went down from 0.3559 in epoch 1 to 0.1236 in epoch 20, while the loss during validation went down from 0.2523 to 0.1303. Both the lines were going down and were very close towards the end, so no overfitting was found in this particular execution.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

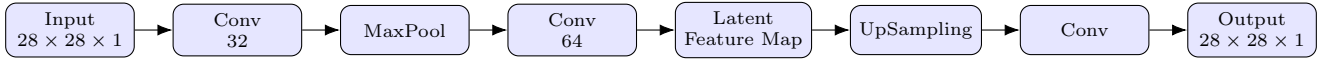
Report:

Metric	Value
Test MSE	0.022110
Test MAE	0.058131
Mean SSIM	0.737771

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```

Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')

```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

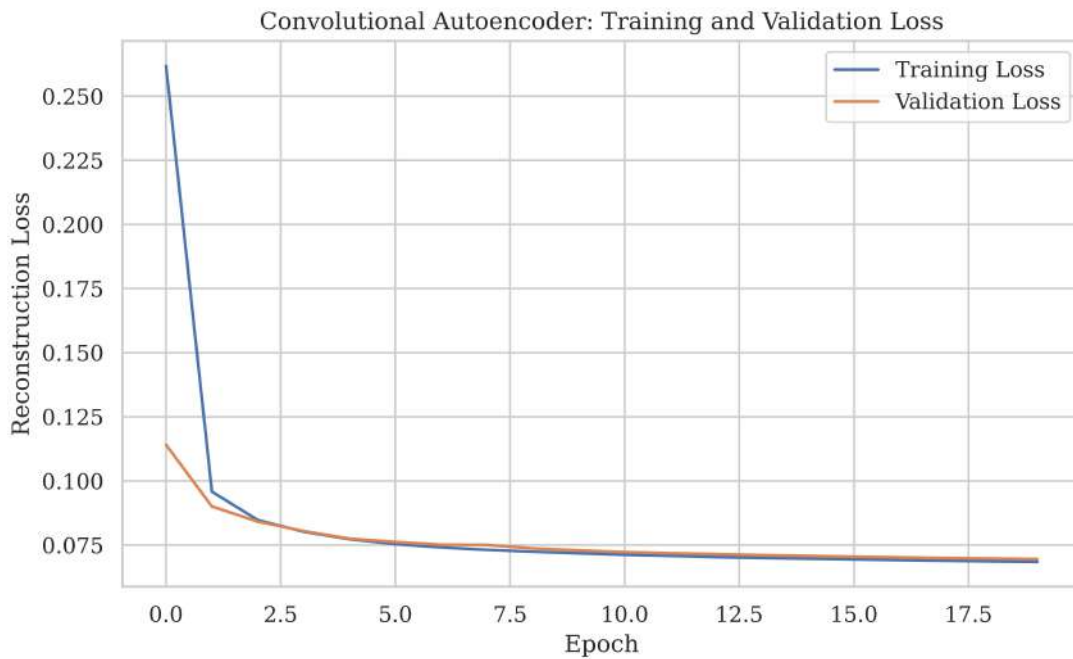


Figure 4: Convolutional autoencoder reconstruction visualization.

Execution finding. Final training loss is 0.0684 and validation loss is 0.0695. Metrics for the test set include MSE = 0.002616, MAE = 0.015094 and mean SSIM = 0.973534. The number of parameters in the network is 74,497 and training took 685.09 seconds.

Inference. The convolutional model maintained local spatial patterns and reconstructed digit structure much more accurately than the fully connected model.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022110	0.058131	0.737771	211,040
Convolutional AE	0.002616	0.015094	0.973534	74,497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.

Required inference:

Discuss how preserving spatial locality through convolution affects image reconstruction.

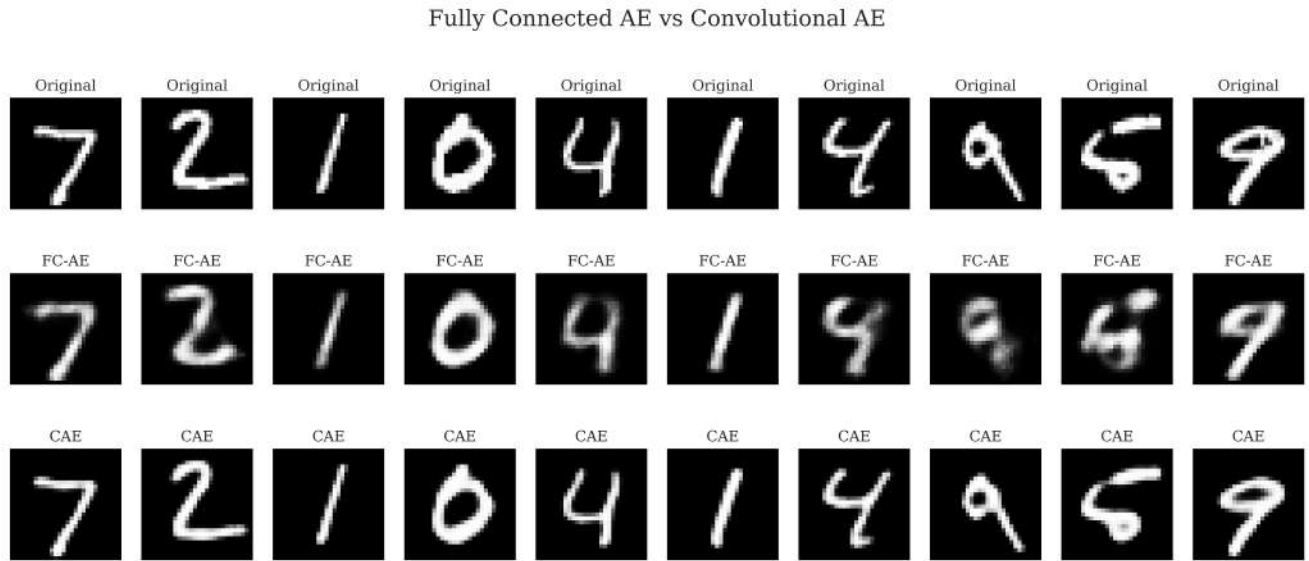


Figure 5: Original image, fully connected AE reconstruction and convolutional AE reconstruction.

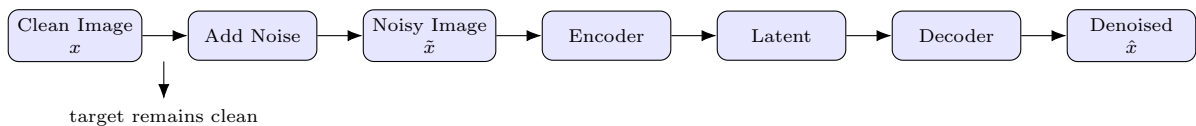
Execution finding. The CAE produced $MSE = 0.002616$, $MAE = 0.015094$ and $SSIM = 0.973534$, while the FC-AE produced 0.022110, 0.058131 and 0.737771 respectively, using fewer parameters (74,497 vs 211,040). This proves the importance of spatial locality and convolutional filters in image reconstruction.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

MSE , MAE , $SSIM$.

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.003855	0.018465	0.954884
0.2	0.005016	0.022132	0.940780
0.3	0.007599	0.029860	0.878881

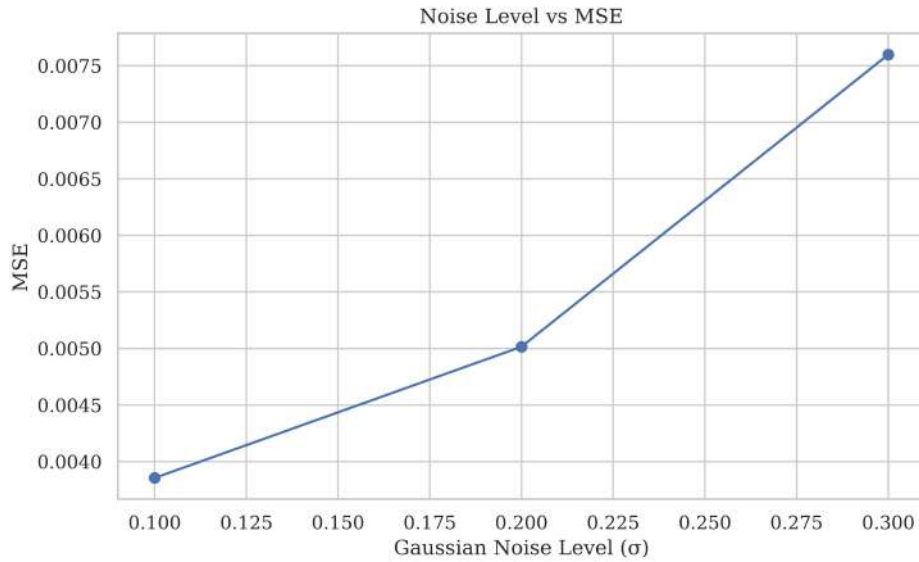


Figure 6: Gaussian noise level versus reconstruction MSE.

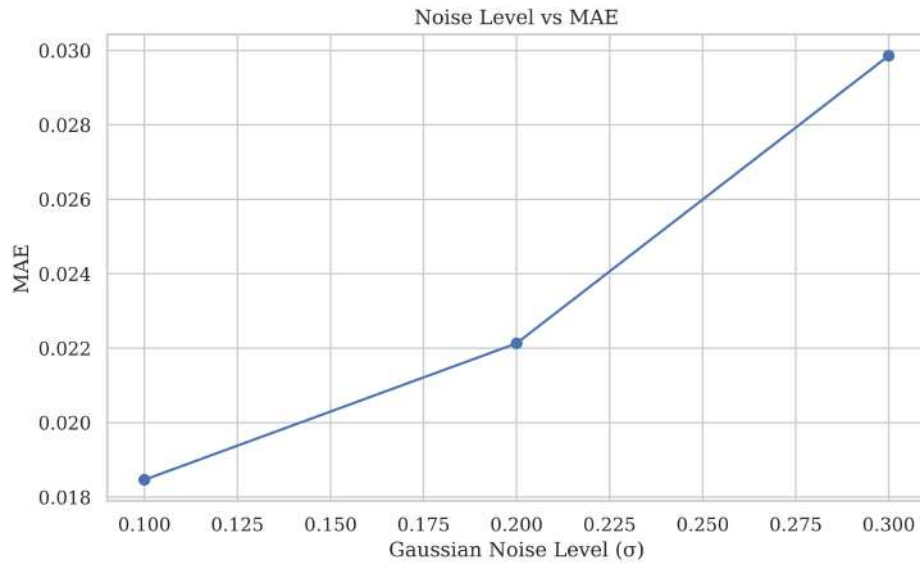


Figure 7: Gaussian noise level versus reconstruction MAE.

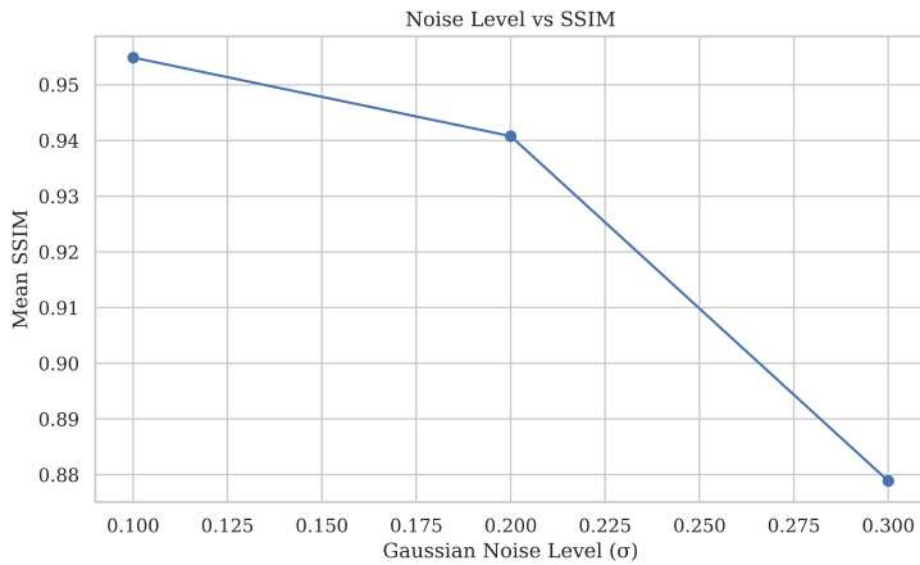


Figure 8: Gaussian noise level versus reconstruction SSIM.

Inference. As the Gaussian noise σ increased from 0.1 to 0.3, the MSE increased from 0.003855 to 0.007599, MAE increased from 0.018465 to 0.029860, and SSIM dropped from 0.954884 to 0.878881. As the amount of corruption increases, the accuracy of recovery becomes increasingly harder but similarity of structure remains.

Required inference:

Describe how higher levels of corruption will impact reconstruction quality and state whether the denoising algorithm can successfully reconstruct the main features of the original image.

Clean vs Noisy vs Denoised Images

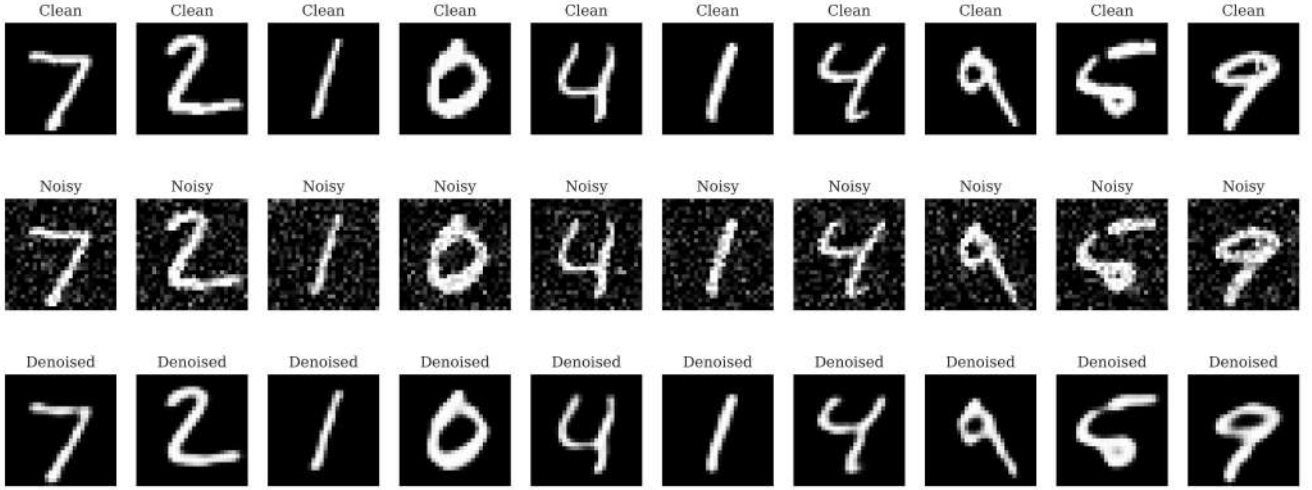


Figure 9: Clean, Gaussian-corrupted and denoised test images for $\sigma = 0.2$.

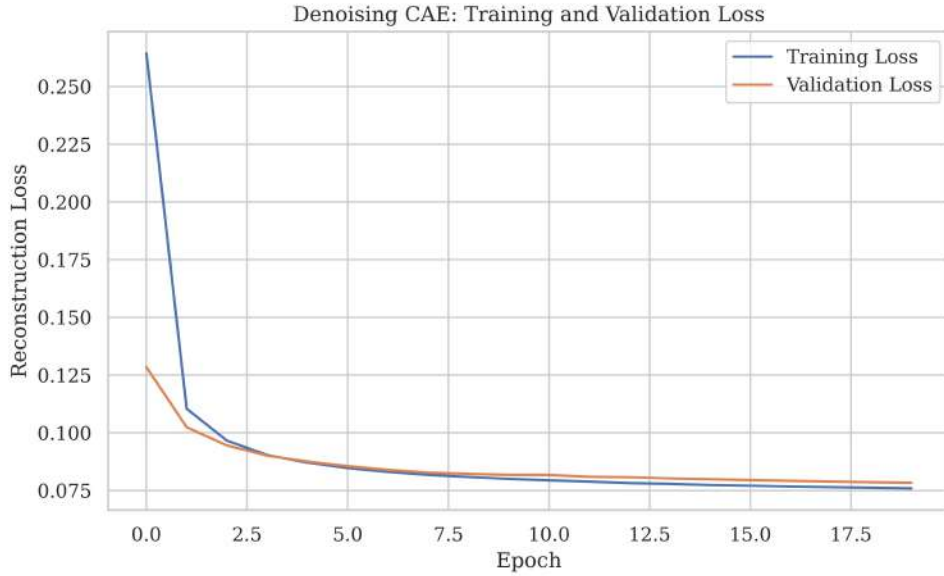


Figure 10: Denoising CAE training and validation loss.

Execution finding. The training loss was 0.0759 and the validation loss was 0.0783. On noise-free test targets, the denoising CAE had an $MSE = 0.004969$, $MAE = 0.022047$ and $SSIM = 0.941478$ using 74,497 weights and 672.30 training time in seconds.

Inference. The denoising model has successfully managed to remove most of the noise added to the image without altering the structure of the digits. This is clear from the high value of SSIM, indicating that the structural information is largely recovered.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

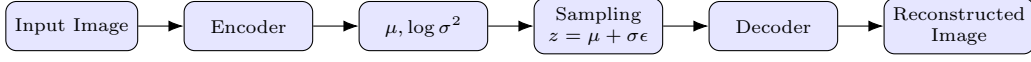
$$q_{\phi}(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

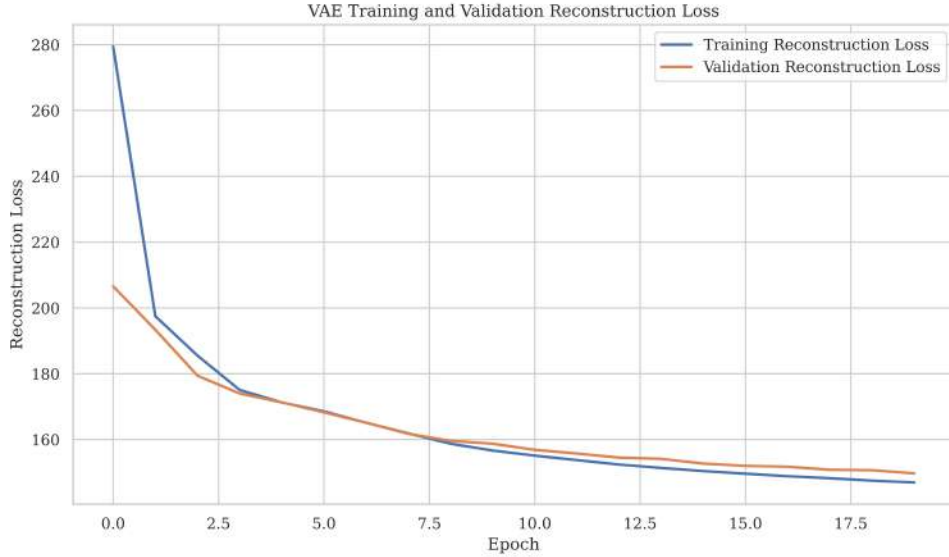


Figure 11: VAE training and validation reconstruction loss.

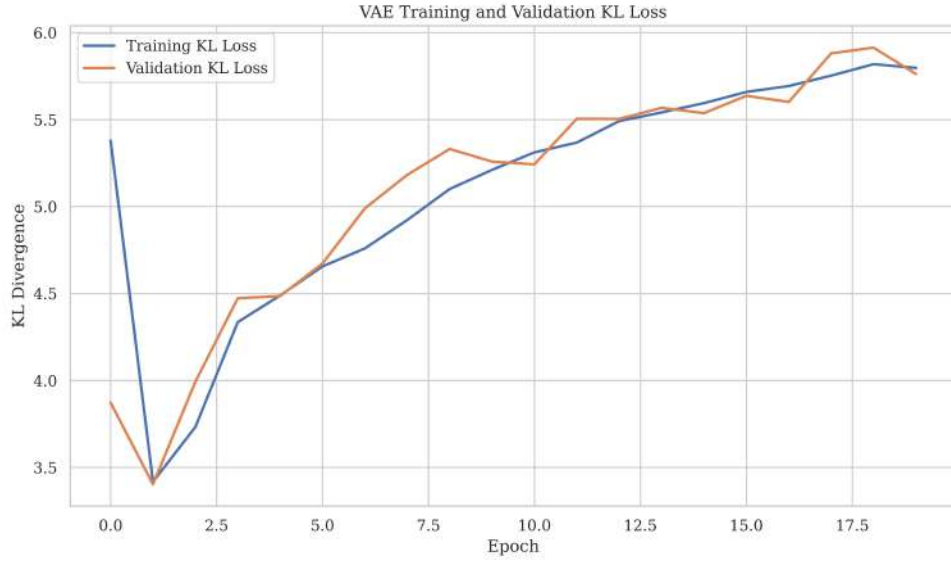


Figure 12: VAE training and validation KL loss.

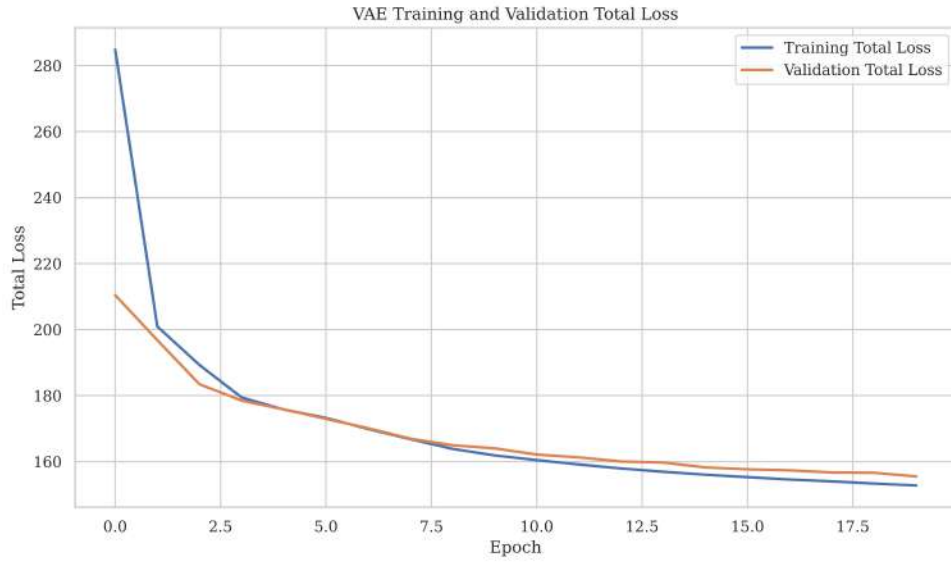


Figure 13: VAE training and validation total loss.

Execution finding. The last held out test loss values were reconstruction loss = 148.990936, KL loss = 5.725443, and total loss = 154.716400. The respective image values were MSE = 0.043799, MAE = 0.102828, and mean SSIM = 0.485724. The VAE had 485,957 parameters and needed 356.05 seconds for training.

Inference. VAE image reconstruction was more smooth but less accurate than that by CAE. It is because of the probabilistic latent constraint, where the model reconstructs the image while regularizing the latent variable distribution to a standard normal distribution at the same time.

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

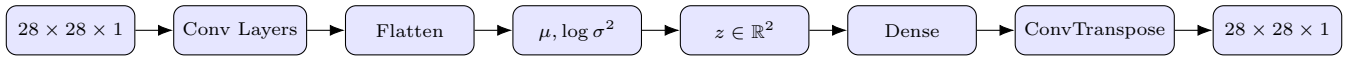
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

Required inference:

Students must analyze whether the samples from the same class lie in similar clusters and whether the samples from different classes overlap.

Original vs VAE Reconstruction

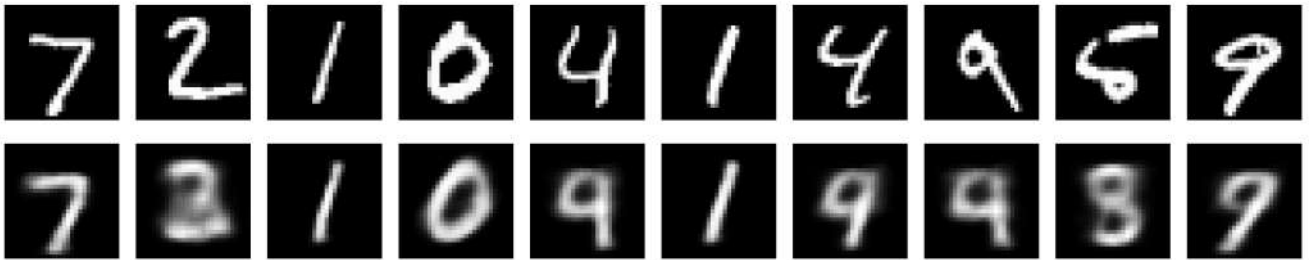


Figure 14: Original test images and VAE reconstructions.

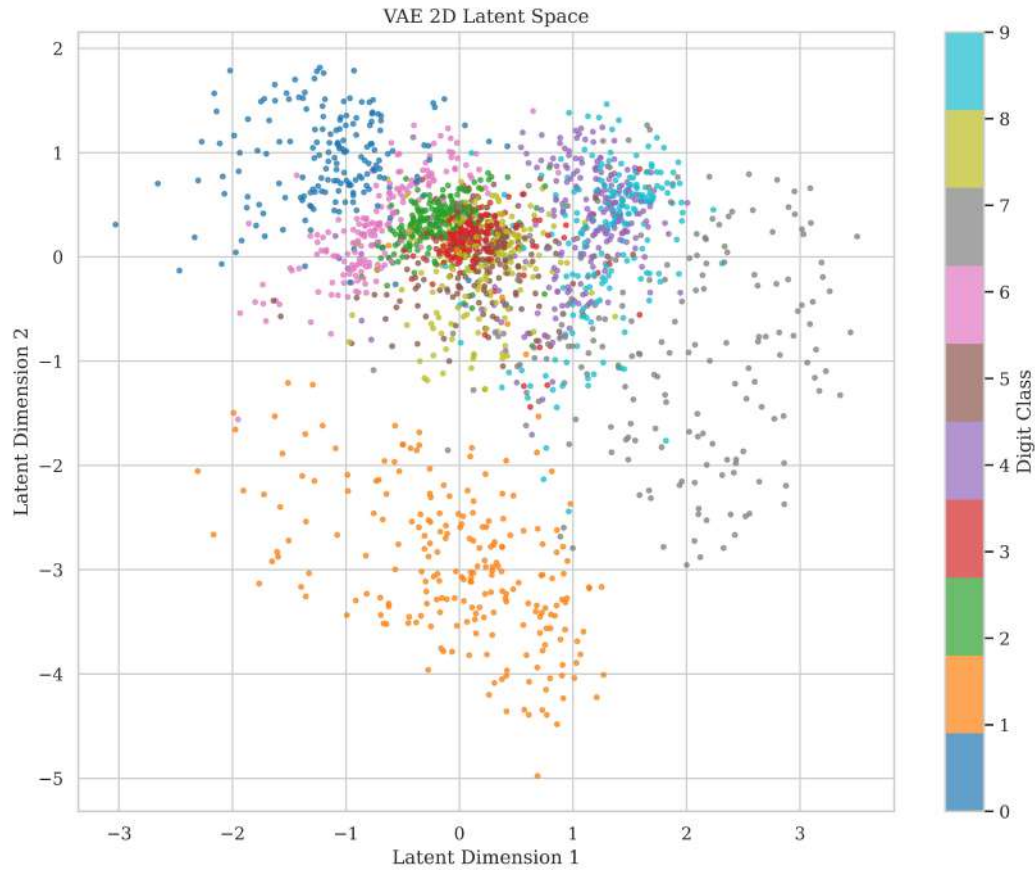


Figure 15: Two-dimensional VAE latent-space visualization colored by digit label.

Inference. The hidden visualization depicts distinct groups for visually similar digits, with some cross-over between digit categories. The two-dimensional format allows one to examine the learned probabilistic structure visually.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

VAE Randomly Generated Images

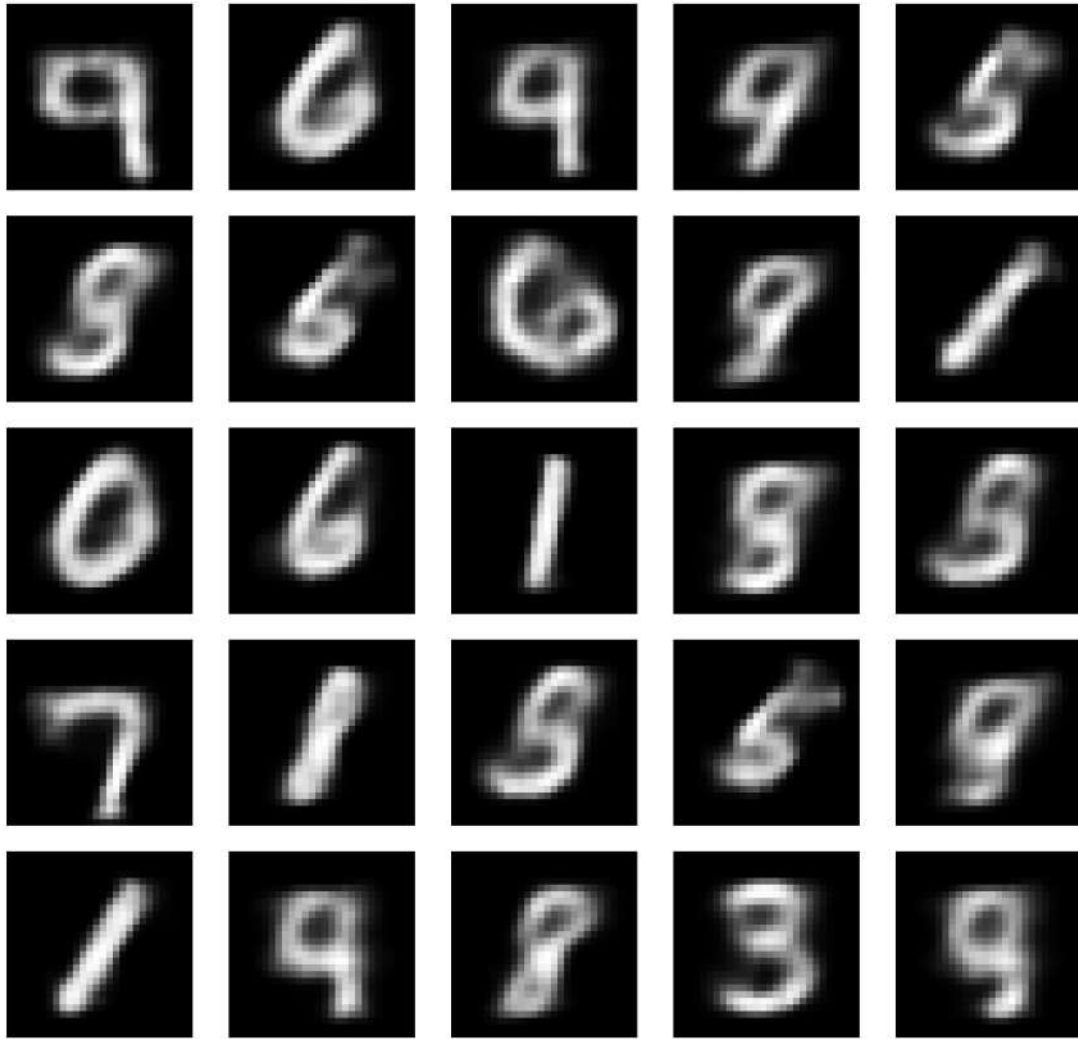


Figure 16: Twenty-five images generated by sampling latent vectors from the standard normal prior.

Inference. The output is similar to handwritten digits that you would find in MNIST, and they clearly vary in shape and stroke formation. Some samples are clearer than others, thus making the difference between generation and reconstruction.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

Required inference:

Identify if the reconstructed image undergoes a smooth transformation when the latent vector varies between two points.

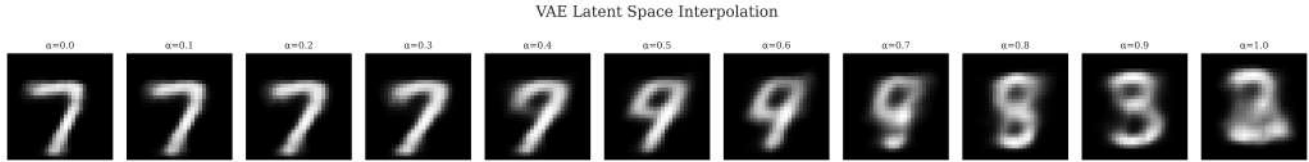


Figure 17: Decoded images along the VAE latent interpolation path from $\alpha = 0$ to $\alpha = 1$.

Inference. There is a gradual transformation of the decoded images from one step to another on the interpolation route. It means that there is a relationship between the neighboring latent points with similar image structure.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022110	0.058131	0.737771	211,040
Convolutional AE	0.002616	0.015094	0.973534	74,497
Denoising CAE	0.004969	0.022047	0.941478	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	148.990936
KL Loss	5.725443
Total Loss	154.716400
Test MSE	0.043799
Test MAE	0.102828
Mean SSIM	0.485724

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

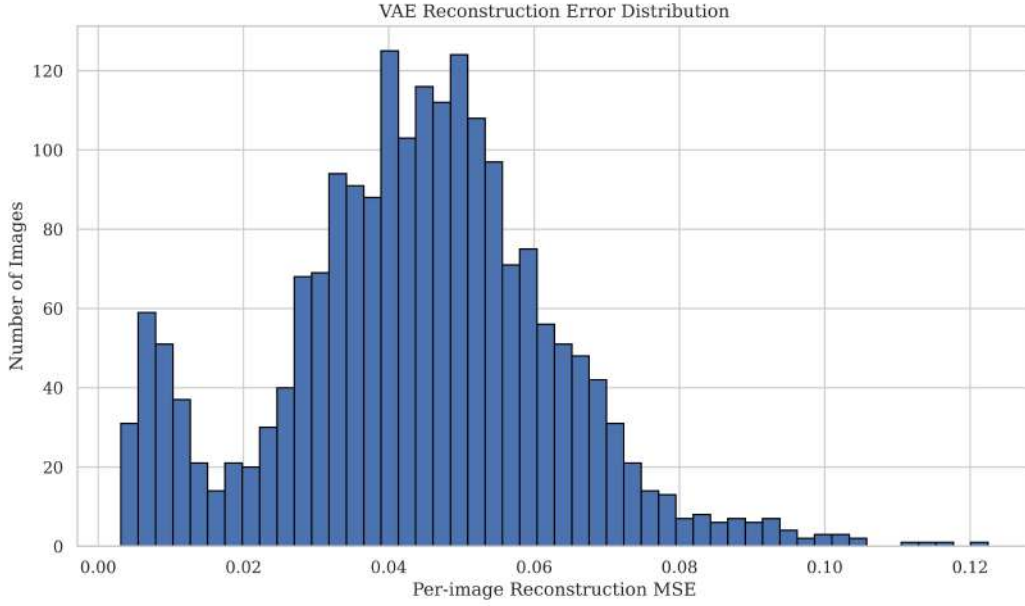


Figure 18: VAE per-image reconstruction-error distribution on the test set.

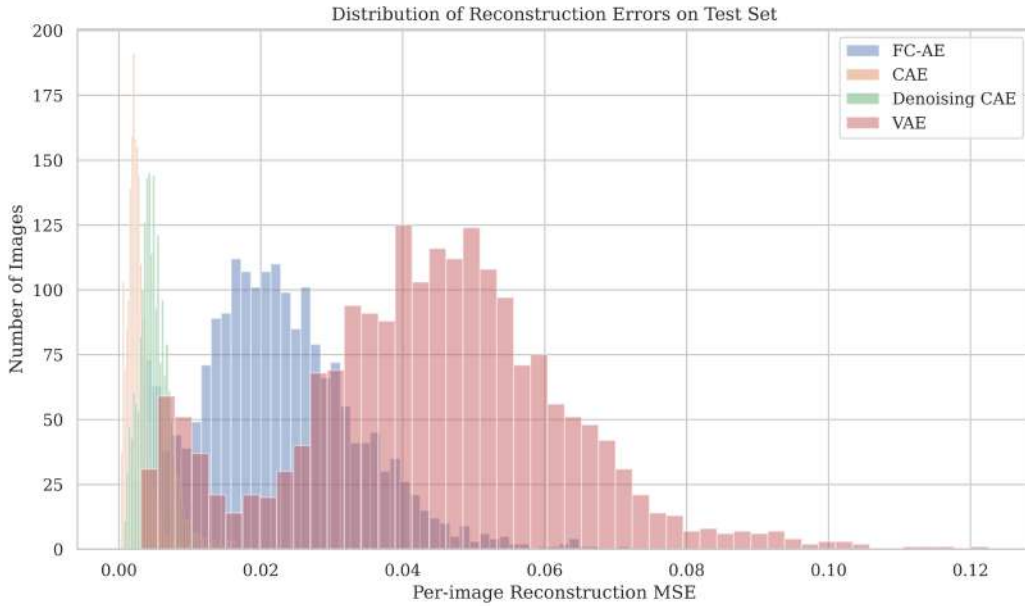


Figure 19: Per-image reconstruction-error distributions for the FC-AE, CAE, denoising CAE and VAE.

Inference. The reconstruction error distributions corroborate the aggregate statistics: The CAE reconstruction error is skewed towards lower numbers, while the VAE reconstruction error distribution is skewed towards higher numbers. Higher reconstruction error samples refer to images whose reconstructed strokes or shapes have a greater difference from the original images.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Sample	Original	Reconstruction
1	Index 1782, digit 8, MSE 0.071842	See Fig. 20
2	Index 654, digit 5, MSE 0.067009	See Fig. 20
3	Index 1801, digit 9, MSE 0.064957	See Fig. 20
4	Index 876, digit 8, MSE 0.064668	See Fig. 20
5	Index 338, digit 8, MSE 0.064561	See Fig. 20

Students should provide a short explanation for why these images may be difficult to reconstruct.

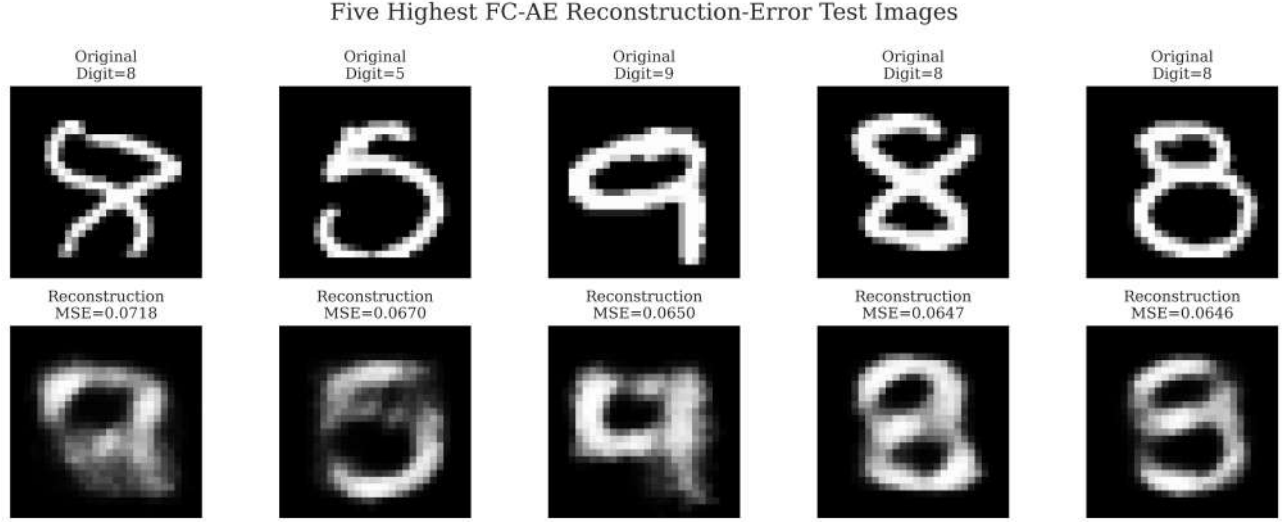


Figure 20: Five test images with the largest FC-AE reconstruction errors and their reconstructions.

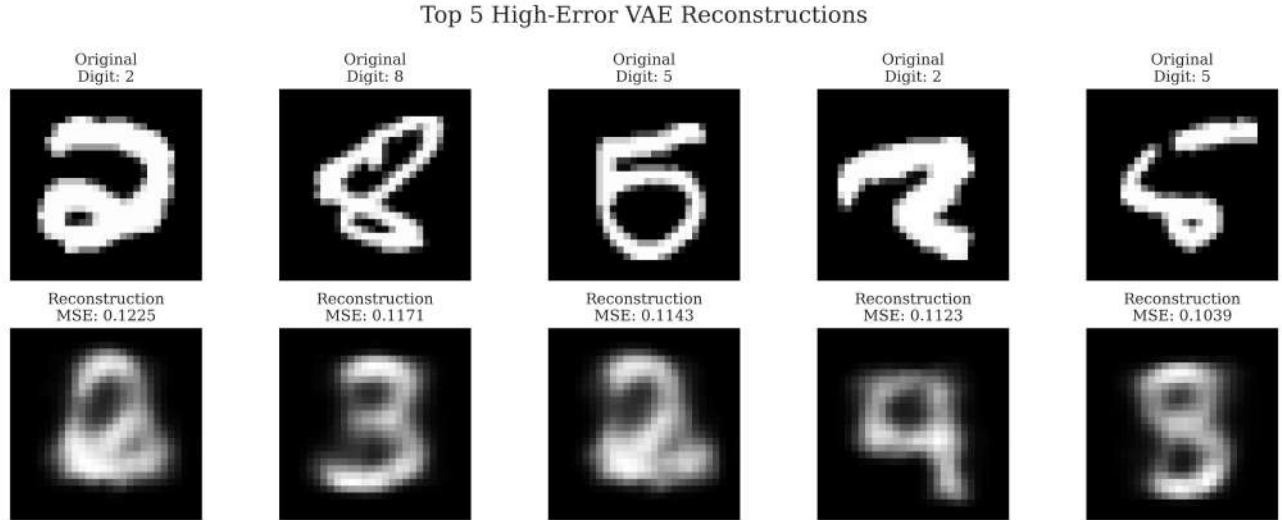


Figure 21: Five high-error examples identified using VAE per-image reconstruction MSE.

Execution finding. For FC-AE, the five biggest mistakes were test indices 1782 (digit 8, MSE = 0.071842), 654 (digit 5, MSE = 0.067009), 1801 (digit 9, MSE = 0.064957), 876 (digit 8, MSE = 0.064668), and 338 (digit 8, MSE = 0.064561). For VAE, the five biggest mistakes were test indices 1352 (digit 2, MSE = 0.122466), 1325 (digit 8, MSE = 0.117107), 1847 (digit 5, MSE = 0.114303), 1790 (digit 2, MSE = 0.112336), and 8 (digit 5, MSE = 0.103874).

Inference. High-error examples may have strange stroke thickness, asymmetric loops, ambiguous boundaries, or handwriting styles that deviate from the dominant examples seen during training. High reconstruction errors are thus very useful in detecting hard-to-see samples.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
FC Autoencoder	0.022110	0.058131	0.737771	211,040	27.21 s
Conv. Autoencoder	0.002616	0.015094	0.973534	74,497	685.09 s
Denoising CAE	0.004969	0.022047	0.941478	74,497	672.30 s
VAE	0.043799	0.102828	0.485724	485,957	356.05 s

The values must be obtained from the student's own execution.

Execution finding. The consolidated results from the notebook are as follows: FC-AE: MSE 0.022110, MAE 0.058131, SSIM 0.737771, 211,040 parameters, 27.21 seconds; Conv-AE: MSE 0.002616, MAE 0.015094, SSIM 0.973534, 74,497 parameters, 685.09 seconds; Denoising CAE: MSE 0.004969, MAE 0.022047, SSIM 0.941478, 74,497 parameters, 672.30 seconds; VAE: MSE 0.043799, MAE 0.102828, SSIM 0.485724, 485,957 parameters, 356.05 seconds.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

Execution-Based Findings

- Raising the latent space size of FC-AE from 2 to 32 dropped the MSE from 0.047606 to 0.021787 and increased SSIM from 0.431449 to 0.739935.
- In contrast, the MSE of CAE was 0.002616 and its SSIM was 0.973534 against 0.022110 and 0.737771 for the FC-AE, illustrating the advantages of spatial locality.
- Raising the level of Gaussian noise from $\sigma = 0.1$ to 0.3 increased MSE and MAE and dropped the SSIM.
- The denoising CAE identified the most dominant structure of the digits in spite of the noise, reaching SSIM 0.941478 at the level of noise $\sigma = 0.2$ where it was trained.
- The VAE created a latent probabilistic representation with parameters μ and $\log \sigma^2$.
- Latent space had structured clusters and overlapping classes but only for the purpose of visualization, and no labels were used.
- Previously generated images had been different MNIST-like characters although there were also some blurred images.
- The path taken during the interpolation was smooth between the two chosen points in latent space.
- Reconstruction error for each image was high when there were noticeable differences between the original strokes and reconstructions.
- FC-AE and CAE employ deterministic latent codes, while VAE uses a probability distribution which makes it possible to sample and generate images directly.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.047606	0.431449	210,130
8	0.030645	0.645589	210,520
16	0.024599	0.708026	211,040
32	0.021787	0.739935	212,080

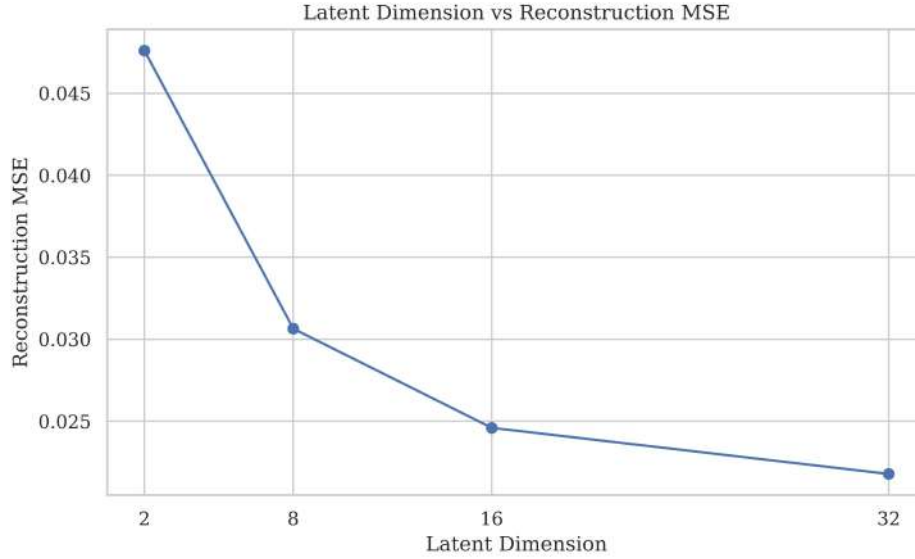


Figure 22: Latent dimension versus reconstruction MSE.

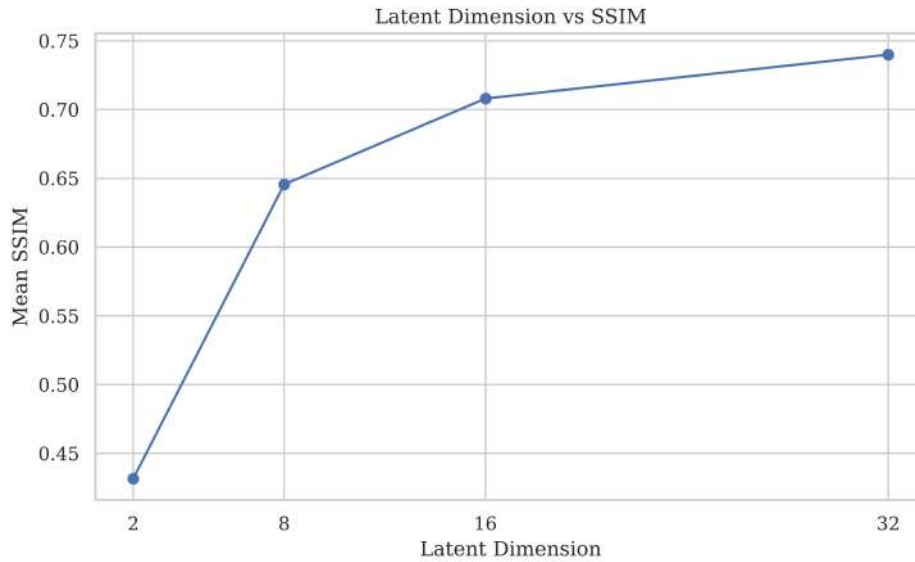


Figure 23: Latent dimension versus mean SSIM.

Inference. A rise in the number of latent dimensions led to lower MSE and greater SSIM. The largest gain was seen from 2 to 8 dimensions and then became less significant. The number of parameters did not change much due to the relatively low number of nodes in the latent layer compared to the rest of the dense layers.

Plot 10: Latent Dimension vs. Reconstruction MSE

Students should explain the trade-off between compression and reconstruction quality.

Finding. FC-AE quality of reconstruction based on MSE values shows that as we increase the number of neurons in the latent layer, quality of reconstruction improves. Indeed, the value of MSE decreases from 0.047606 at $d_z = 2$ to 0.021787 at $d_z = 32$. It means that the higher compression rate, the higher amount of information is lost. The smaller the compression rate, the less information loss occurs.

28. Discussion Questions

1. What is an autoencoder?
2. What is the purpose of the encoder?
3. What is the purpose of the decoder?
4. What is a latent representation?
5. Why is the latent representation usually smaller than the input?
6. What happens when the latent dimension is too small?
7. What happens when the latent dimension is increased?
8. Why can a convolutional autoencoder preserve spatial information more effectively?
9. Differentiate an autoencoder and a denoising autoencoder.
10. Why is the clean image used as the target in denoising?
11. What happens when the noise level is increased?
12. Why are MSE, MAE and SSIM useful for image reconstruction?
13. What is a Variational Autoencoder?
14. How does a VAE differ from a conventional autoencoder?
15. Why does a VAE learn a distribution instead of a single latent vector?
16. What is the reparameterization trick?
17. Why is ϵ sampled from a standard normal distribution?
18. What is the role of KL divergence in the VAE loss?
19. What is the prior distribution used in the VAE?
20. Why is a two-dimensional latent space useful for visualization?
21. What does latent-space interpolation demonstrate?
22. Why can a VAE generate new images?
23. Why might some VAE-generated images be unclear?
24. What is the difference between reconstruction and generation?
25. Why should test images remain unseen during training?

Answers

1. Autoencoders are neural networks that learn to encode the input into the latent representation and decode the latent representation back to a reconstruction of the input.
2. The encoder converts the input image into the latent representation, which contains information useful for the reconstruction.
3. The decoder decodes the latent representation back into the image space and generates the reconstruction of the input.
4. Latent representation is the internal compressed representation obtained by the encoder.
5. The smaller latent representation forms a bottleneck forcing the network to keep the useful information rather than copying the input.
6. The larger latent dimension results in the information leakage and leads to poor reconstruction if the latent dimension is too large.
7. The larger latent dimension provides more information through the bottleneck and results in better reconstruction as seen from the latent dimensions 2 to 32.
8. Convolutions operate on the local neighborhoods with shared filters, while a fully connected representation does not.
9. Conventional autoencoders reconstruct their inputs while denoising autoencoders take a corrupted input and reconstruct the clean image.
10. Since the clean image is the desired result, we can use it as the ground truth and force the neural net to eliminate corruption.
11. As noise level is raised, reconstruction becomes more challenging; in our case, MSE and MAE values became higher, but SSIM became lower.
12. MSE focuses on squared deviations of pixels; MAE estimates the average absolute deviation; SSIM quantifies the structural similarity between two images.
13. Variational autoencoder (VAE) is an autoencoder where the encoder learns the parameters of the latent distribution and the decoder can reconstruct and generate samples from that distribution.
14. In general autoencoders create a deterministic latent vector, while variational autoencoders learn a distribution and sample a latent vector from it.
15. Distribution regularizes the latent space and allows us to sample new latent vectors for generation.
16. Reparameterization trick is $z = \mu + \sigma \odot \epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$.
17. Sampling ϵ from a standard Gaussian distribution ensures a separation of the stochastic part from the learned parameters and gives a fixed reference distribution for reparameterization.
18. KL-divergence loss helps the learned latent distribution to remain close to the selected prior.

19. The prior distribution used in this experiment is a standard multivariate Gaussian distribution $\mathcal{N}(0, I)$.
20. A two-dimensional latent space can be visualized directly and therefore the cluster structure, overlap, and interpolation path become easily interpretable.
21. Latent space interpolation shows how small changes of the latent vector cause gradual changes in the decoded images.
22. A VAE can create new images because it is possible to sample latent vectors from the learned prior distribution and pass them through the decoder.
23. Generated images may not always be clear because it is possible that the sampled latent vector decodes to an atypical area of the learned distribution.
24. Reconstruction is decoding of a latent vector corresponding to a certain input image, while generation begins from sampling a new latent vector.
25. By making test images unavailable, we get an unbiased performance assessment for images that were not used during optimization of the model.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

Execution Status and Findings

Code-based results based on conducted experiments:

1. The CAE with latent channel capacity increased from 4 to 32 will be able to examine the compression-reconstruction tradeoff, since increased latent dimensionality means more feature data passing through the bottleneck.
2. Salt-and-pepper noise is an example of a complementing corruption because it changes selected pixels to extreme values, not to continuous perturbations as Gaussian corruption does.
3. Using two different models trained for $\sigma = 0.1$ and $\sigma = 0.3$ will show whether the model that is specialized in one corruption can reconstruct the clean image under another corruption.
4. Transposed convolutional decoder will be used to test whether the model can reconstruct the spatial output using learned upsampling and not the UpSampling2D layer.
5. Latent dimensions 2 and 8 of the VAE allow us to compare a low- and high-dimensional latent space.
6. Raising the weight of KL-loss emphasizes the need for the learned latent distribution to be closer to the prior, resulting in an explicit reconstruction versus regularization trade-off.
7. The generation of 100 samples provides a broader qualitative analysis of the diversity of samples in VAE compared to the 25 samples requirement.
8. Interpolation analysis determines if the latent transitions are smooth if endpoints come from different digit regions.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014.
3. Pascal Vincent et al., “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, “MNIST Handwritten Digit Database.”
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>